

funnelling

Marco Servetto 

School of Engineering and Computer Science, Victoria University of Wellington, New Zealand

Colin S. Gordon 

School of Computer & Information Sciences, Drexel University, United States of America

Abstract

...

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Funding Colin S. Gordon: US NSF CISE Award CCF-2007582

Acknowledgements I want to thank ...

1 Introduction

Java includes a useful feature called *method-local inner classes*, which is surprisingly subtle. Like anonymous inner classes, this is a way for a method in a Java program to declare a class whose details are defined locally, entirely within the method, when the identity of the class is not needed outside the method's scope. However, these local classes are named, which allows them to be used with other language features exploiting nominal types. For example, one can cast objects to the method-local class type.

Unfortunately, as Servetto et al. [6] pointed out, these features interact unsoundly with Java generics. In generic methods (or methods of generic classes), a method-local inner class `MyFoo` which extends a type that depends on generic parameters is treated the same regardless of whether it was allocated with the same generic parameters, or different generic parameters. Servetto et al. probed the behaviour of JVM languages which supported the same feature (Scala and Kotlin) and found that these had the same unsoundness problem for some use cases, but did flag concerns in other cases. However this was not systematically investigated.

In this paper we clearly articulate the problem they observed, characterize the key properties that lead to the issue, and demonstrate both dynamic and static approaches to resolve the issue in future languages. The dynamic approach is a specific interpretation of type tests with reified generics, providing guidance to language implementors on a subtle design choice available in that space. The static approach is a general solution for tracking when a nominal object type *declaration* effectively captures generic parameter instantiations from its dynamic context.

Figure 1 demonstrates the issue. Line 3 declares a method-local inner class `MyFoo`, which depends on the generic parameter instantiation for `X` when the method is invoked. The `main` method invokes `m` twice. Line 8 invokes it for `Integers`. Since the `other` argument for that call is not a `MyFoo`, the first invocation returns a `MyFoo` which is a `Foo<Integer>`. This result is then passed back in to the second call to `m` on Line 9. This time, the `instanceof` check succeeds, because `other` in this case *is* a `MyFoo`, so the method (successfully) casts the `other` argument to `MyFoo`. Since `MyFoo` is declared as extending `Foo<X>`, this is a valid result for the function. Thus the `Foo<Integer>` that was passed as the second argument on Line 9 is returned as a `Foo<String>`.

Clearly this is wrong, because there are now two aliased variables of incompatible types; consequently, the later code fails (we discuss the failure point later). The description of

```

1 public class Broken {
2     static <X> Foo<X> m(Object other, final X x){
3         class MyFoo extends Foo<X>{ public X x(){ return x; } }
4         if(!(other instanceof MyFoo)){ return new MyFoo(); }
5         return ((MyFoo)other); // cast passes with no warnings
6     }
7     public static void main(String[] a) {
8         Foo<Integer> r1= Broken.<Integer>m("anything",5);
9         Foo<String> r2= Broken.<String>m(r1,"Hi");
10        System.out.println(r1.x()); //5
11        System.out.println(r2.x()); //Throws ClassCastException: ..
12    }
13 }
14 class Foo<X>{ public X x() {return x();} }

```

■ **Figure 1** Subtly broken Java code

45 execution above employed some (here, intentionally) sloppy reasoning to parallel the checks
 46 in the Java compiler.

- 47 ■ The `instanceof` check could have been statically flagged.
- 48 ■ The cast could have been statically flagged.
- 49 ■ The `instanceof` could have returned false in the second invocation, since it was used on a
 50 `MyFoo` from a different generic parameter instantiation
- 51 ■ The case could have failed, again because it was used on a `MyFoo` from a different generic
 52 parameter instantiation

53 Unfortunately two key factors lead to Java accepting this code and only incidentally discover-
 54 ing the problem later:

- 55 1. Java implements generics by erasure. So the runtime representation of `MyFoo` doesn't
 56 differ across the two calls. No information is kept around to distinguish the two classes of
 57 `MyFoo`. This rules out the dynamic solutions.
- 58 2. The Java compiler does not treat `MyFoo` as generic. Attempts to cast to, or use `instanceof`
 59 with `Foo<Integer>` are statically flagged specifically because erasure means the generic
 60 parameters cannot be dynamically checked. But because `MyFoo` *syntactically* lacks generic
 61 parameters of its own, the compiler treats it as *non-generic* and allows the `instanceof`
 62 and cast without complaint.

63 Subtler still is the actual cause of the failure. The two `println` calls actually dispatch to
 64 different overloads: the first to a version accepting an `Object`, and the second to the more
 65 specific overload expecting a `String`. It is in fact the passage of a value from a generic API
 66 to the specific overload which causes the Java compiler to insert a `checkcast` in the generated
 67 bytecode.

68 It is tempting to accept this as a quirk of Java, which is already known to have other
 69 unsoundness [2]. However this problem is broader in both scope and impact, as we show in the
 70 rest of the paper. This issue affects all statically typed languages on the JVM, including Scala
 71 and Kotlin. It also propagates to other compilation targets for JVM languages, including
 72 Scala Native and GraalVM's Native Image support.

73 In the remainder of this paper:

- 74 ■ We detail the breadth of the problem (surveying forms that use class generics rather than
 75 methods generics, closures, and other manifestations), and how it is handled by various
 76 existing language implementations

```

1  class Object{};
2  template<typename X> class Foo : public Object {
3      public: X x() { return x(); } };
4  template<typename X>
5  Foo<X>* m(Object* other, const X* x) {
6      class MyFoo : public Foo<X> { public: X x() { return x; } };
7      MyFoo* d = dynamic_cast<MyFoo*>(other);
8      if(d == nullptr) { return new MyFoo(); }
9      return d;
10 }

```

■ **Figure 2** C++ equivalent of Figure 1.

- 77 ■ We develop three variants of a core language modeling common implementation choices
78 for generics, and show that only one catches the problem dynamically before allowing
79 multiple incompatible types for aliased variables.
- 80 ■ We develop a new type system mechanism for systematically tracking would-be implicit
81 dependence of a method-local class on generic parameters, and show that it soundly
82 prevents the problem across any of the runtime semantics.

2 Scope of the Problem

84 This problem arises in essentially all JVM languages, all of which must interact with Java
85 and have therefore chosen to also support method-local classes.

2.1 Generic Reification

87 A natural suspicion is that this is “simply” a problem with generic erasure, that of course
88 a language that cannot distinguish generic instantiations of a class at runtime will have
89 difficulties of this sort. There is some truth to this: Java is permitting a cast that it cannot
90 actually check due to type erasure. However there is more to it than that: Java will eagerly
91 flag code such as `(List<String>)(new LinkedList<Integer>())`. So the language does intend
92 to warn about uses of uncheckable casts. And while reified generics do provide enough
93 information to check casts dynamically, only certain forms of reification make correct casts
94 straightforward.

95 In a language which implements parametric polymorphism by templating / heterogeneous
96 compilation (C++, C#, or early experiments with generic Java [1]), the check is straight-
97 forward: each instantiation of `m` with different generic parameters is compiled separately,
98 resulting in separate class declarations for each instantiation. Consider reinterpreting Figure
99 1 in C++ as in Figure 2. `gcc` reports for Line 7:

```

100 <source>:7:16: error: cannot 'dynamic_cast' 'other' (of type 'class Object*')
101 to type 'class m<Integer>(Object*, const Integer*):MyFoo*' (source type is not polymorphic)
102     7 |         MyFoo* d = dynamic_cast<MyFoo*>(other);
103         |                               ~~~~~~

```

not perfect, because this is C++ giving a static warning, not catching the error dynam-
ically...

Expressions	e	$::=$	$x \mid e.\langle Ts \rangle m(es) \mid e.[T]\langle Ts \rangle m(es) \mid e.f \mid \text{new } C\langle Ts \rangle(es)$ $\mid \text{new } C\langle Fs \rangle \text{ extends } D\langle Ts' \rangle(es)\{Ms\}$ $\text{if } (e \text{ instanceof } T? x) \{e\} \text{ else } \{e\}$
Patterns	P	$::=$	$C \mid C\langle Ts \rangle$
Types	T	$::=$	$C\langle Ts \rangle \mid X$
Funnels	F	$::=$	$G \mid X \text{ funnel}$
Ground Types	G	$::=$	$C\langle Gs \rangle$
Type Env.	Γ	$::=$	$\epsilon \mid \Gamma, T x$
Bounds	Δ	$::=$	$\epsilon \mid \Delta, X \text{ extends } T$
Declaration	Dec	$::=$	$\text{class } C\langle \Delta \rangle \text{ extends } C'\langle Ts \rangle \{ Ms Ss \}$
(Field) Slots	S	$::=$	$T f$

$D \text{ declared}$	$\frac{\text{class } C\langle \Delta \rangle \text{ extends } C'\langle Ts \rangle \{ Ms Ss \} \in \text{Decls}}{\text{class } C\langle \Delta \rangle \text{ extends } C'\langle Ts \rangle \{ Ms Ss \} \text{ declared}}$
----------------------	---

$\frac{\text{class } C\langle \Delta \rangle \text{ extends } C'\langle Ts \rangle \{ Ms Ss \} \text{ declared} \quad \text{new } C\langle Fs \rangle \text{ extends } D\langle \overline{T} \rangle(\bar{e})\{\overline{M}\} \in Ms}{\text{new } C\langle Fs \rangle \text{ extends } D\langle \overline{T} \rangle(\bar{e})\{\overline{M}\} \text{ declared}}$
$\frac{\text{new } C\langle Fs \rangle \text{ extends } D\langle \overline{T} \rangle(_)\{Ms\} \text{ declared} \quad \text{new } C'\langle Fs' \rangle \text{ extends } D'\langle \overline{T}' \rangle(\bar{e})\{\overline{M}\} \in Ms}{\text{new } C\langle Fs \rangle \text{ extends } D\langle \overline{T}' \rangle(\bar{e})\{\overline{M}\} \text{ declared}}$

■ **Figure 3** Syntax and Declaration Checking

105 Notice that the error message uses the instantiation of the method signature as a
 106 scope for a declaration of `MyFoo`. But heterogeneous compilation has subtle costs and wide-
 107 ranging consequences for language design, not just implementation, and complex performance
 108 consequences [5, 4]. Even if this does fix the problem, it is not universally appropriate.

109 An alternative, common conception of generic reification is to simply save the type
 110 parameters along with the single implementations of classes, for it to be checked when needed.
 111 But note again that `MyFoo` has no generic parameters of its own. So a shallow check with this
 112 variant of reified generics is insufficient — each `MyFoo` would need to contain a representation
 113 of, or reference to, its parent type's generic parameters. And because this phenomenon is
 114 not limited to a single step in the inheritance hierarchy, the cast must traverse and compare
 115 a substantial amount of data — not a mere comparison of virtual table pointers. This again
 116 has multiple implementation approaches, neither ideal (chains of parent type instantiations
 117 hurt cache locality, inlining data can bloat small instances).

118 So there remains a need to articulate a solution to this problem where generics are
 119 implemented via erasure.

120 ah, we need to check Flix, where maybe we can cook up something roughly like this?

121 **3 A Core Language for Funnelling**

122 Substitution replaces every occurrence of a type variable, *including* the ones in the method
 123 signatures of a method-local class. A funnelled variable is the only form of shadowing of type
 124 variables that we allow; the domain of the substitution is disjoint from the free type variables

$$\boxed{e \rightarrow e'}$$

$$\begin{array}{c}
\text{SIMPLEFIELDFOUND} \frac{\text{class } C\langle Bs \rangle \text{ extends } C'\langle Ts \rangle \{ Ms \ T_1 f_1 \dots T_n f_n \} \in \text{Decls} \quad f_i \in f_1 \dots f_n}{\text{new } C\langle Gs \rangle(v_1 \dots v_n \ vs).f_i \rightarrow v_i} \\
\\
\text{SIMPLEFIELDSUPER} \frac{\text{class } C\langle Bs \rangle \text{ extends } C'\langle Ts \rangle \{ Ms \ T_1 f_1 \dots T_n f_n \} \in \text{Decls} \quad f \notin f_1 \dots f_n}{\text{new } C\langle Gs \rangle(v_1 \dots v_n \ vs).f \rightarrow \text{new } C'\langle Ts[Bs = Gs] \rangle(v_s).f} \\
\\
\text{INNERFIELDFOUND} \frac{}{\text{new } C\langle Gs \rangle \text{ extends } C'\langle Gs' \rangle(v_s)\{Ms\}.f \rightarrow \text{new } C'\langle Gs' \rangle(v_s).f} \\
\\
\text{IFTHENERASED} \frac{v \in \{\text{new } C\langle Gs \rangle(_), \text{new } C\langle Gs \rangle \text{ extends } _ \langle _ \rangle(_)\{ _ \} \} \quad \epsilon \vdash C\langle Gs \rangle <: D\langle _ \rangle}{\text{if } (v \text{ instanceof } D\langle Ts \rangle \ x) \ \{e_1\} \text{ else } \{e_2\} \rightarrow e_1[x = v]} \\
\\
\text{IFELSEERASED} \frac{v \in \{\text{new } C\langle Gs \rangle(_), \text{new } C\langle Gs \rangle \text{ extends } _ \langle _ \rangle(_)\{ _ \} \} \quad \epsilon \not\vdash C\langle Gs \rangle <: D\langle _ \rangle}{\text{if } (v \text{ instanceof } D\langle Ts \rangle \ x) \ \{e_1\} \text{ else } \{e_2\} \rightarrow e_2} \\
\\
\text{SIMPLEADDSUPER} \frac{v = \text{new } C\langle Gs \rangle(_)}{v.\langle Gs' \rangle m(vs) \rightarrow v.[C\langle Gs \rangle]\langle Gs' \rangle m(vs)} \\
\\
\text{SIMPLEFOUND} \frac{\text{class } C\langle Bs \rangle \text{ extends } _ \langle _ \rangle \{ Ms \ Ss \} \in \text{Decls} \quad \langle Bs' \rangle T_0 \ m(T_1 \ x_1, \dots T_n \ x_n) \ \{ \text{return } e_0 \} \in Ms}{e = e_0[Bs, Bs' = Gs, Gs'][\text{this}, x_1 \dots x_n = v_0 \dots v_n]} \\
\frac{}{v_0.[C\langle Gs \rangle]\langle Gs' \rangle m(v_1 \dots v_n) \rightarrow e} \\
\\
\text{INNERFOUND} \frac{v_0 = \text{new } _ \langle _ \rangle \text{ extends } _ \langle _ \rangle(_)\{Ms\} \quad \langle Bs' \rangle T_0 \ m(T_1 \ x_1, \dots T_n \ x_n) \ \{ \text{return } e_0 \} \in Ms \quad e = e_0[Bs' = Gs'][\text{this}, x_1 \dots x_n = v_0 \dots v_n]}{v_0.\langle Gs' \rangle m(v_1 \dots v_n) \rightarrow e} \\
\\
\text{INNERSUPER} \frac{v_0 = \text{new } _ \langle _ \rangle \text{ extends } C\langle Gs \rangle(_)\{Ms\} \quad m \notin Ms}{v_0.\langle Gs' \rangle m(v_1 \dots v_n) \rightarrow v_0.[C\langle Gs \rangle]\langle Gs' \rangle m(v_1 \dots v_n)} \\
\\
\text{SEARCHSUPER} \frac{\text{class } C\langle Bs \rangle \text{ extends } _ \langle _ \rangle \{ Ms \ Ss \} \in \text{Decls} \quad m \notin Ms}{v.[C\langle Gs \rangle]\langle Gs' \rangle m(vs) \rightarrow v.[C'\langle Ts[Bs = Gs] \rangle]\langle Gs' \rangle m(vs)} \quad \text{CTX} \frac{e \rightarrow e'}{\mathcal{E}[e] \rightarrow \mathcal{E}[e']}
\end{array}$$

■ **Figure 4** Operational Semantics

$$\begin{array}{c}
\boxed{\Delta; \Gamma \vdash e : T} \quad \text{SUBS} \quad \frac{\Delta; \Gamma \vdash e : T_1 \quad \Delta \vdash T_1 <: T_2 \quad \Delta \vdash T_2 \text{ Ok}}{\Delta; \Gamma \vdash e : T_2} \quad \text{VAR} \quad \frac{}{\Delta; \Gamma \vdash x : \Gamma(x)} \\
\\
\text{DECLFIELD} \quad \frac{\Delta; \Gamma \vdash e : C(\overline{T}) \quad \text{class } C(\Delta') \text{ extends } C'(\overline{T}') \{ M s T_1 f_1 \dots T_n f_n \} \in \text{Decls}}{\Delta; \Gamma \vdash e.f_i : T_i[\Delta' = \overline{T}]} \\
\\
\text{CALL-TOP} \quad \frac{\Delta; \Gamma \vdash e_0 : C_0(\overline{T}_0) \quad \text{class } C_0(\Delta_0) \text{ extends } C'(\overline{T}') \{ \overline{M} \overline{S} \} \text{ declared } \langle \Delta_1 \rangle T_0 m(T_1 x_1, \dots T_n x_n) \{ _ \} \in \overline{M} \quad T'_i = T_i[\Delta_0, \Delta_1 = \overline{T}_0, \overline{T}_1] \quad \Delta; \Gamma \vdash e_1 : T'_1 \dots \Delta; \Gamma \vdash e_n : T'_n \quad \Delta \vdash \overline{T}_1 \text{ Ok}}{\Delta; \Gamma \vdash e_0.\langle \overline{T}_1 \rangle m(e_1, \dots, e_n) : T'_0} \\
\\
\text{CALL-LOCAL} \quad \frac{\Delta; \Gamma \vdash e_0 : C_0(\overline{T}_0) \quad \text{new } C_0(\Delta_0) \text{ extends } C'(\overline{T}') \{ \overline{M} \overline{S} \} \text{ declared } \langle \Delta_1 \rangle T_0 m(T_1 x_1, \dots T_n x_n) \{ _ \} \in \overline{M} \quad T'_i = T_i[\Delta_0, \Delta_1 = \overline{T}_0, \overline{T}_1] \quad \Delta; \Gamma \vdash e_1 : T'_1 \dots \Delta; \Gamma \vdash e_n : T'_n \quad \Delta \vdash \overline{T}_1 \text{ Ok}}{\Delta; \Gamma \vdash e_0.\langle \overline{T}_1 \rangle m(e_1, \dots, e_n) : T'_0} \\
\\
\text{CALL-LB-RUNTIME} \quad \frac{\Delta; \Gamma \vdash e_0 : T \quad \text{topLevelClass}(T) \quad \Delta \vdash T <: C_0(\overline{T}_0) \quad \text{class } C_0(\Delta_0) \text{ extends } C'(\overline{T}') \{ \overline{M} \overline{S} \} \text{ declared } \langle \Delta_1 \rangle T_0 m(T_1 x_1, \dots T_n x_n) \{ _ \} \in \overline{M} \quad T'_i = T_i[\Delta_0, \Delta_1 = \overline{T}_0, \overline{T}_1] \quad \Delta; \Gamma \vdash e_1 : T'_1 \dots \Delta; \Gamma \vdash e_n : T'_n \quad \Delta \vdash \overline{T} \text{ Ok} \Delta \vdash \overline{T}_1 \text{ Ok}}{\Delta; \Gamma \vdash e_0.[T] \langle \overline{T}_1 \rangle m(e_1, \dots, e_n) : T'_0} \\
\\
\text{NEW-TOP} \quad \frac{\text{class } C(\Delta') \text{ extends } C'(\overline{T}') \{ \overline{M} T_1 f_1 \dots T_n f_n \} \text{ declared } \quad \Delta; \Gamma \vdash \text{new } C'(\overline{T}')[\Delta' = \overline{T}](\overline{e}) : _ \quad \Delta \vdash \overline{T} \text{ Ok}}{\Delta; \Gamma \vdash \text{new } C(\overline{T})(e_1 \dots e_n \overline{e}) : C(\overline{T})} \\
\\
\text{NEW-LOCAL} \quad \frac{\Delta; \Gamma \vdash \text{new } C'(\overline{T}')(\overline{e}) : _ \quad \begin{array}{l} \Delta' \vdash C'(\overline{T}) \text{ Ok} \quad F_i = X_i \text{ funnel} \Rightarrow T_i = X_i \quad F_i = G_i \Rightarrow T_i = G_i \\ \Delta' \vdash C(T_1 \dots T_n) \text{ Ok} \quad \Delta' = \{ X \text{ extends } T \mid X \text{ extends } T \in \Delta \wedge \exists i. F_i = X \text{ funnel} \} \\ \forall i. F_i = X_i \text{ funnel} \Rightarrow X_i \text{ extends } _ \in \Delta \quad \text{what about } X_i \text{ extends } Y \text{ where } Y \text{ isn't funnelled?} \\ \Gamma' = \Gamma \setminus \text{FTV}(T_1 \dots T_n) \end{array} \quad \Delta'; \Gamma', \text{this} : C(T_1 \dots T_n) \vdash M_1 \text{ Ok} \dots \Delta'; \Gamma', \text{this} : C(T_1 \dots T_n) \vdash M_k \text{ Ok}}{\Delta; \Gamma \vdash \text{new } C(F_1 \dots F_n) \text{ extends } C'(\overline{T}')(\overline{e}) \{ M_1 \dots M_k \} : C(T_1 \dots T_n)} \\
\\
\text{IF-INSTANCEOF} \quad \frac{\Delta; \Gamma \vdash e_0 : T_0 \quad \Delta \vdash T \text{ Ok} \quad \Delta; \Gamma, T x \vdash e_1 : T' \quad \Delta; \Gamma \vdash e_2 : T'}{\Delta; \Gamma \vdash \text{if } (e_0 \text{ instanceof } T x) \{ e_1 \} \text{ else } \{ e_2 \} : T'} \quad \boxed{\Gamma; \Delta \vdash M \text{ Ok}} \\
\\
\text{METHOD-OK} \quad \frac{\Delta, \Delta'; \Gamma, x_1 : T_1, \dots, x_n : T_n \vdash e : T_0 \quad \Delta, \Delta' \vdash T_0 \text{ Ok} \dots \Delta, \Delta' \vdash T_n \text{ Ok}}{\Delta; \Gamma \vdash \langle \Delta' \rangle T_0 m(T_1 x_1 \dots T_n x_n) \{ \text{return } e; \} \text{ Ok}}
\end{array}$$

■ Figure 5 Typing

$$\begin{array}{c}
\boxed{\Delta \vdash T \text{ Ok}} \qquad \text{WF-VAR} \quad \frac{X \text{ extends } _ \in \Delta}{\Delta \vdash X \text{ Ok}} \\
\\
\text{WF-OUTER} \quad \frac{\Delta \vdash T_1 \text{ Ok} \dots \Delta \vdash T_n \text{ Ok} \quad \text{class } C \langle X_1 \text{ extends } T'_1, \dots, X_n \text{ extends } T'_n \rangle \text{ extends } _ \langle _ \rangle \{ _ _ \} \in \text{Decls} \quad \Delta \vdash T_i <: T'_i[\overline{X} = \overline{T}]}{\Delta \vdash C \langle T_1 \dots T_n \rangle \text{ Ok}} \\
\\
\text{WF-INNER} \quad \frac{\Delta \vdash T_1 \text{ Ok} \dots \Delta \vdash T_n \text{ Ok} \quad \text{class } C \langle \Delta_C \rangle \text{ extends } _ \langle _ \rangle \{ _ _ \} \in \text{Decls} \quad \langle \Delta_M \rangle _ m(_) \{ \text{return } e; \} \in M_{SC} \quad \text{new } C_0 \langle F_1 \dots F_n \rangle \text{ extends } _ \langle _ \rangle (_) \{ _ \} \text{ occurs in } e \quad F_i = X_i \text{ funnel} \Rightarrow X_i \text{ extends } T'_i \in \Delta_C, \Delta_M \wedge \Delta \vdash T_i <: T'_i[\overline{X} = \overline{T}] \quad F_i = G_i \Rightarrow T_i = G_i}{\Delta \vdash C_0 \langle T_1 \dots T_n \rangle \text{ Ok}} \\
\\
\boxed{\Delta \vdash T <: T'} \qquad \text{SUB-REFL} \quad \frac{}{\Delta \vdash T <: T} \\
\\
\text{SUB-INHERIT} \quad \frac{\text{class } C \langle \Delta_0 \rangle \text{ extends } C' \langle \overline{T'} \rangle \{ _ _ \} \in \text{Decls} \quad \Delta \vdash C' \langle \overline{T'} \rangle [\Delta_0 = \overline{T}] <: T''}{\Delta \vdash C \langle \overline{T} \rangle <: T''} \\
\\
\text{SUB-INHERIT-LOCAL} \quad \frac{\text{new } C \langle F_s \rangle \text{ extends } C' \langle \overline{T'} \rangle (_) \{ _ \} \text{ declared} \quad \Delta \vdash C' \langle \overline{T'} \rangle [\text{dom}(F_s) = \overline{T}] <: T''}{\Delta \vdash C \langle \overline{T} \rangle <: T''} \\
\\
\text{SUB-VAR} \quad \frac{X \text{ extends } T \in \Delta \quad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}
\end{array}$$

■ **Figure 6** Kinding and Subtyping

$$\begin{array}{lcl}
x[\overline{X} = \overline{T}] & = & x \\
e.\langle \overline{T'} \rangle m(\overline{e'})[\overline{X} = \overline{T}] & = & e[\overline{X} = \overline{T}].\langle \overline{T'}[\overline{X} = \overline{T}] \rangle m(\overline{e'}[\overline{X} = \overline{T}]) \\
e.\langle \overline{T'} \rangle \langle \overline{T''} \rangle m(\overline{e'})[\overline{X} = \overline{T}] & = & e[\overline{X} = \overline{T}].\langle \overline{T'}[\overline{X} = \overline{T}] \rangle \langle \overline{T''}[\overline{X} = \overline{T}] \rangle m(\overline{e'}[\overline{X} = \overline{T}]) \\
e.f[\overline{X} = \overline{T}] & = & e[\overline{X} = \overline{T}].f \\
\text{new } C \langle \overline{T'} \rangle (\overline{e})[\overline{X} = \overline{T}] & = & \text{new } C \langle \overline{T'}[\overline{X} = \overline{T}] \rangle (\overline{e}[\overline{X} = \overline{T}]) \\
\text{if } (e_0 \text{ instanceof } T' \ x) \{ e_1 \} \text{ else } \{ e_2 \} [\overline{X} = \overline{T}] & = & \text{if } (e_0[\overline{X} = \overline{T}] \text{ instanceof } T'[\overline{X} = \overline{T}] \ x) \{ e_1[\overline{X} = \overline{T}] \} \text{ else } \{ e_2[\overline{X} = \overline{T}] \} \\
\text{new } C \langle \overline{F} \rangle \text{ extends } C' \langle \overline{T'} \rangle (\overline{e}) \{ \overline{M} \} [\overline{X} = \overline{T}] & = & \text{new } C \langle \overline{F}[\overline{X} = \overline{T}] \rangle \text{ extends } C' \langle \overline{T'}[\overline{X} = \overline{T}] \rangle (\overline{e}[\overline{X} = \overline{T}]) \{ \overline{M}[\overline{X} = \overline{T}] \} \\
\\
X' \text{ funnel}[\overline{X} = \overline{T}] & = & X' \text{ funnel} \quad \text{if } X' \notin \overline{X} \\
X_i \text{ funnel}[\overline{X} = \overline{T}] & = & T_i \\
G[\overline{X} = \overline{T}] & = & G \\
\\
\langle \Delta \rangle T_0 \ m(\overline{T} \ x) \{ \text{return } e; \} [\overline{X} = \overline{T'}] & = & \langle \Delta[\overline{X} = \overline{T'}] \rangle T_0[\overline{X} = \overline{T'}] \ m(\overline{T}[\overline{X} = \overline{T'}] \ x) \{ \text{return } e[\overline{X} = \overline{T'}]; \} \\
(X' \text{ extends } T')[\overline{X} = \overline{T}] & = & X' \text{ extends } T'[\overline{X} = \overline{T}] \\
\\
C \langle \overline{T'} \rangle [\overline{X} = \overline{T}] & = & C \langle \overline{T'}[\overline{X} = \overline{T}] \rangle \\
X'[\overline{X} = \overline{T}] & = & X' \quad \text{if } X' \notin \overline{X} \\
X_i[\overline{X} = \overline{T}] & = & T_i
\end{array}$$

■ **Figure 7** Substitution of type variables

of the substituted types and from the bounds in scope, so no renaming is ever needed.

...

The rule NEW-LOCAL is the heart of funneling, including subtle management of type environments. The management of type environments in NEW-LOCAL is subtle. The rule constructs a restricted set of type bounds and variable bindings reflecting only the explicitly-funnelled type variables: Δ' includes only the explicitly-funnelled type variables and Γ' is restricted to just those variables free in the result type (which must be well-kinded under Δ'). However, the rule still uses the *full* ambient environments for checking certain subexpressions. This is because some $e \in \bar{e}$ may initially contain uses of *non*-funnelled type variables (e.g., an explicit call), but will reduce to a value which no longer depends on such type variables. This is why the first antecedent checks the effective supertype instance under $\Delta; \Gamma$, but the second antecedent ensures the resulting type of that expression satisfies the funnelling constraints by checking kinding under the more restrictive Δ' .

...

NEW-LOCAL-JAVA

$$\frac{\Delta; \Gamma \vdash \text{new } C' \langle \overline{T'} \rangle (\bar{e}) : _ \quad \Delta \vdash C' \langle \overline{T} \rangle \text{ Ok} \quad \Delta; \Gamma, \text{this} : C \langle \rangle \vdash M_1 \text{ Ok} \dots \Delta; \Gamma, \text{this} : C \langle \rangle \vdash M_k \text{ Ok}}{\Delta; \Gamma \vdash \text{new } C \langle \rangle \text{ extends } C' \langle \overline{T'} \rangle (\bar{e}) \{M_1 \dots M_k\} : C \langle \rangle}$$

139

briefly note that Java allows extra, “fresh” generic parameters in local classes

140

call out that we’re allowed to declare return types that mention an inner class; inner classes are “public”, and our approach works even in this case, the more restrictive case of Java is naturally covered

141

articulate early: It’s not just about generic reification: just checking type parameters aren’t enough, as our MyFoo cases have no generic params. You need to **deeply** check reified generics, you need to check not just the base type but also check the choice(s) of parent types from the context into the runtime check.

142

4 (Un)Soundness of Method Local Classes

143

With our language primitives in place, we now consider type safety of 4 variants of our language, with varied typing rules and cast semantics. We schematically denote these languages by $\zeta(C, T)$ for our language with cast dynamics C and type rule T . As a result we consider:

1. $\zeta(\text{erased}, \text{Java})$, which models Java’s generic erasure in case semantics, and Java’s type checking
2. $\zeta(\text{reified}, \text{Java})$, which models Java’s type checking *but* reifies generics and checks their instantiation as part of casts (similar to C#)
3. $\zeta(\text{erased}, \text{funnelled})$, which applies funnelling to a model with generic erasure
4. $\zeta(\text{reified}, \text{funnelled})$, which applies funnelling to a model with reified generics that are checked during casts

5 Miscellaneous

155

//—— MARCO: (6th time) This is not about generic erasure In a language with reified generics, if the language still decides that MyFoo is not generic, no impact of those reified

157

generics is going to be seen in the case (MyFoo)other MyFoo extends Foo<X> is 'as generic' as String extends Comparable<String> A cast to String, would not look in the generics of Comparable<String> A cast to MyFoo, would not look in the generics of Foo<X> If subtyping has (nominal-) 'reflexivity' then String <: String (no need to look at the generics in Comparable<String>, available or not. A language with implicit 'type scope' that could conceptually add to the 'name' when types are identified in the type system could change this. Eg: scala foo.Bar is a type and foo.Bar != beer.Bar //existential types on constants/values Broken.<X>m.MyFoo //existential types on generic-instantiated methods //----- COLIN: This is ambiguous: with reified generics, there are two implementation strategies, which yield different behaviours: 1. MyFoo allocated as a Foo<String> gets a different class table from MyFoo allocated as a Foo<Integer>. In this case, the cast will fail 2. All MyFoo will get the same inherited class table, but keep a separate field with the choice of X for Foo<X>, used for inherited methods. In this case, there are *two possible cast implementations*: a. Cast checks only vtable, and cast succeeds. Arguably a compiler that "treats MyFoo as non-generic" could lean towards this b. Cast checks both vtable and the runtime representation of the type parameter of Foo. Arguably a compiler that preserves this for inherited methods from Foo to work correctly would be likely to check this (since it would be checked in a cast to Foo<Integer>).

//-----

This is a problem in other languages (cite cite) as shown in greater details in Rel Work.

Now, assume we have a readable small step reduction over a version of FJ rich enough to express the problem. We will assume: - Simple generics - A syntax like anonymous inner classes, but where we give them names. We can call them named inner classes. This imports the concept of method local inner classes within an expression based calculus. //our model will also need casts + instanceof

```

class Any{}
class A{ A onlyA()-> new A() }
class B{ B onlyB()-> new B() }
class Foo<X> extends Any{ X x() {return x();} }
public class Broken {
  <X> Foo<X> m(Any other, X x) ->
    if(other instanceof MyFoo)
      then ((MyFoo)other)
      else new MyFoo() extends Foo<X>{ X x(){ return x; } } //Bad
}
main:
  new Broken().<B>m(new Broken().<A>m(new Any(),new A()),new B()).x().onlyB();

```

```

class Foo<X> extends Any{ X x() {return x();} }
public class Broken {
  <X> Foo<X> m(Any other, X x) ->
    if(other instanceof MyFoo<X>) //unchecked instanceof (ill typed in our core)?
      then ((MyFoo<X>)other)//unchecked cast (ill typed in our core)?
      else new MyFoo<X>() extends Foo<X>{ public X x(){ return x; } } //good
}

```

Note how funnelling purpose is to insert the generics. Without funnelling, MyFoo is a non generic type, so the question about generic reification does not even arise.

small step reduction

01: new Broken().m(new Broken().<A>m(new Any(),new A()),new B()).x().onlyB();

```

210 02: new Broken().<B>m(
211     if(new Any() instanceof MyFoo<X>)
212         then (MyFoo<A>) new Any()
213         else new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
214         ,new B()).x().onlyB();
215 03: new Broken().<B>m(
216     if(false)
217         then (MyFoo<A>) new Any()
218         else new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
219         ,new B()).x().onlyB();
220 04: new Broken().<B>m(
221     new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
222     ,new B()).x().onlyB();
223 05: (if(new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } } instanceof MyFoo<B>)
224     then (MyFoo<A>) new MyFoo<A>() extends Foo<A>{ public A x()->new A() }
225     else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
226 06a: (if(false)//reified gen
227     then (MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
228     else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
229 07a: (new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
230 08a: new B().onlyB()
231 09a: new B();//success and different semantic
232 06b: (if(true)//Java
233     then (MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
234     else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()
235 07b: ((MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }).x().onlyB()
236 08b: (new MyFoo<A>() extends Foo<A>{ public A x()->new A() }).x().onlyB()
237     //assuming cast and instanceof behave consistently (unchecked)
238 09b: new A().x().onlyB()
239 10b: new A().onlyB() //stuck!!
240 6c: (if(err)//compile or at least run time
241     then ((MyFoo<B>) new MyFoo<A>() extends Foo<A>{ public A x(){ return new A(); } }
242     else new MyFoo<B>() extends Foo<B>{ public B x(){ return new B(); } }).x().onlyB()

243 // Maybe above we want MyFoo:Any<A> and MyFoo:Any<B>?
244
245 public class Broken {
246     static <X> Object m(Object other, final X x, Function<X,String> f){
247         class MyFoo{ public X x(){ return x; } }
248         if(!(other instanceof MyFoo)){ return new MyFoo(); }
249         MyFoo local= (MyFoo)other;//cast pass with no warnings
250         System.out.println("Pass with " + x);
251         System.out.println("old one was "+f.apply(local.x()));//throws ClassCastException
252         return local;
253     }
254     public static void main(String[] a) {
255         Object r1= Broken.<Integer>m("anything",5,i->" "+i);
256         Broken.<String>m(r1,"Hi",s->s);
257     }
258 }
259

```

```

260
261 public class Broken {
262     static <X> Object m(Object other, final X x, Function<X,String> f){
263         class MyFoo{ public X x(){ return x; } }
264         if(!(other instanceof MyFoo)){ return new MyFoo(); }
265         MyFoo local= (MyFoo)other;//cast pass with no warnings
266         System.out.println("Pass_with_" + x);
267         System.out.println("old_one_was_"+f.apply(local.x()));//throws ClassCastException
268         return local;
269     }
270     public static void main(String[] a) {
271         Object r1= Broken.<Integer>m("anything",5,i->" "+i);
272         Broken.<String>m(r1,"Hi",s->s);
273     }
274 }
275

```

```

276
277 public class Broken {
278     static <X> Object m(Object other, final X x, Function<X,String> f){
279         class MyFoo{ public X x(){ return x; } }
280         if(other instanceof MyFoo local){ //new instanceof+introduce var (pass with no warnings)
281             System.out.println("Pass_with_" + x); //sneaky 'local' is MyFoo<Integer> even when this is Broken.<String>
282             System.out.println("old_one_was_"+f.apply(local.x()));//throws ClassCastException
283         }
284         return new MyFoo();//sneaky it is MyFoo<Integer>
285     }
286     public static void main(String[] a) {
287         Object r1= Broken.<Integer>m("anything",5,i->" "+i);
288         Broken.<String>m(r1,"Hi",s->s);
289     }
290 }
291

```

292 // TODO: could talk about doing this with modern Java type-switch

```

293
294 // Compiles without error, fails, on jdk 25.0.1
295 import java.util.function.Function;
296 class Broken {
297     static <X> Object m(Object other, final X x, Function<X,String> f){
298         class MyFoo{ public X x(){ return x; } }
299         switch (other) {
300             case MyFoo local: // implicit unchecked cast
301                 System.out.println("Pass_with_" + x); //sneaky 'local' is MyFoo<Integer> even when this is Broken.<String>
302                 System.out.println("old_one_was_"+f.apply(local.x()));//throws ClassCastException (on the return)
303                 break;
304             default:
305                 break;
306         }
307         if(other instanceof MyFoo local){ //new instanceof+introduce var (pass with no warnings)
308             System.out.println("Pass_with_" + x); //sneaky 'local' is MyFoo<Integer> even when this is Broken.<String>
309             System.out.println("old_one_was_"+f.apply(local.x()));//throws ClassCastException
310         }
311         return new MyFoo();//sneaky it is MyFoo<Integer>
312     }
313     public static void main(String[] a) {
314         Object r1= Broken.<Integer>m("anything",5,i->" "+i);
315         Broken.<String>m(r1,"Hi",s->(s.length()+"")); //Broken.<String>m(r1,"Hi",s->s);

```

```

316     }
317 }
318
319
320 //Example that breaks in a NEW WAY in the bytecode
321 // https://godbolt.org/z/sMnPG1TE
322 import java.util.function.Function;
323 class Broken {
324     static <X> Object m(Object other, final X x, Function<X,Integer> f){
325         class MyFoo{ public X x(){ return x; } }
326         switch (other) {
327             case MyFoo local:
328                 System.out.println("Pass with " + x); //sneaky 'local' is MyFoo<Integer> even when this is Broken
329                 System.out.println("old one was "+f.apply(local.x()));//throws ClassCastException
330                 break;
331             default:
332                 break;
333         }
334         if(other instanceof MyFoo local){ //new instanceof+introduce var (pass with no warnings)
335             System.out.println("Pass with " + x); //sneaky 'local' is MyFoo<Integer> even when this is Broken
336             System.out.println("old one was "+f.apply(local.x()));//throws ClassCastException
337         }
338         return new MyFoo(); //sneaky it is MyFoo<Integer>
339     }
340     public static void main(String[] a) {
341         Object r1= Broken.<Integer>m("anything",5,i->i);
342         Broken.<String>m(r1,"Hi",s->s.length());
343     }
344 }
345 // Above seems to fail but without explicit checkcast, failing one of the virtual method calls
346

```

With investigation in bytecode and trying both `s->s.length()` and `String::length` we think that in order to save on bytecode size, our JVM environment under test adds casts in all lambda calls so that method references lambda calls do not need specialized bytecode (eg `MyInterface a=String::length`)

Marco had this idea that casts may be avoided by using dynamic dispatch returning this. Can not work, since method local class is private, there is no way an external interface can mention it to provide the cast method. Of course, the method local is private to the body (this means also the method signature can not use it) This means no `self:Foo` method, nor `self:T` method of a generic class eventually instantiating `T=Foo`.

This means the core language DO NEED `if(e instanceof T x) else`
 //nested funnelling: `new A<X>() B<X> f()-> new B<X>() new A<X>() <K>B<X,K> f()-> new B<X,K>()`

using 'extends' for declarations in top level classes and generic methods using 'funnel' for named inner classes `new A<X funnel Any>() B<X> f()-> new B<X funnel Any>()`
`new A<X funnel Any>() <K extends Any> B<X,K> f()-> new B<X funnel Any,K funnel Any>()` or, if the bounds are not repeated `new A<X funnel>() <K extends Any> B<X,K> f()-> new B<X funnel,K funnel>()` Very funky syntax: we get `new B<X funnel Any,K funnel Any>()` but, during reduction, this can become we get `new B<Person,K funnel Any>()` where we have a mixture of concrete types and generic funnelling

Global class definition uses `class C<X extends Bar>..` named method local class uses `new C<X funnel>()..` normal new: `new C<Ts>()`

```

368 e ::= x | e.m(es) | e.f | new C<Ts>(es) | new C<Fs>(){Ms} | if (e0 instanceof T x){ e1 }else{ e2
369 T ::= C<Ts> | X
370 F ::= C<CCs> | X funnel // Important, can not be T | X funnel, otherwise we could have an X not 'f
371 CC ::= C<CCs> //types with zero free type variables
372 B ::= X extends T // Need T, not F, to be able to do things like (Java) class Bar<X,Y extends X>
373 Dec ::= class C<Bs>{ Ms Ss} //assume standard constructor
374 S ::= T x; //field/slot
375 M ::= Sig; | Sig{ return e; }
376 Sig ::= <Bs> T m(x1:T1,..xn:Tn)
377 G ::= x1:T1..xn:Tn

```

```

378 //we do not need Object to be able to write 'extends Object' //this is exactly as the
379 distinction of checked and unchecked exceptions: formally it evaporates in syntactic sugar
380 where all is checked //and all methods 'throws Error, RTE' // Can assume the user provides
381 their own Object / Any type Note: an use of X is just X, a declaration of X is X extends T;
382 a funnelling of X is X funnel Program is just many Dec plus an e typable in empty MARCO
383 TODO: typechecking Decs;Bs;G|-e:T Decs|-Dec:ok Decs;T|-M:ok e -> e' Note, there are
384 multiple semantics/ts: e instanceof CC //always clear e instanceof T with T not of form CC
385 would not typecheck

```

```

386 e -> e' for Java with warn->err: just have CC in instanceof Java with magic refined cast:
387 just have T in instanceof (since reduction will replace (thus remember) generics Java with
388 warn->off: AT RUN TIME instanceof only checks base class, ignores generic parameters
389 AT COMPILE TIME, instanceof can be err if the full type (including generics) is not
390 potentially correlated)?? unobvious consider interfaces In prose, should explain that we won't
391 flag cast of some type Foo to some type Bar, even though Java sometimes flags unrelated
392 casts

```

```

393 <X extends List<Y funnel>
394 class Foo<X> class Getter<X funnel> <Y extends List<X> X> X.getFirst(Y y) y.getOrNull(0,null)
395
396 <Y> .. return foo<Y,MyX>(); <Y,X extends List<Y> foo() return Foo<X funnel>()...
397 X ;

```

6 Related Work

It is not uncommon to have “intentional unsoundness” in industrial languages, such as Java’s covariant array subtyping, or TypeScript’s covariant method subtyping. However these are intentionally designed into the language to improve ease of use, and in most cases (like Java’s array handling) problematic uses are detected at runtime (e.g., Java checks upon storing into an array that the new entry value is a subtype of the array’s original content type).

However, unintentional unsoundness in industrial type systems is rarer. The most notable example is Amin and Tate’s discovery [2] that Java and Scala’s handling of generic type constraints was rendered unsound by the fact that `null` is treated as inhabiting all reference types, making all type constraints (even nonsensical ones) satisfiable. Servetto et al. [6] first pointed out the issue publicly, though in the Q&A session after the talk, multiple Java designers were unsurprised — as Gilad Bracha put it, “Once I saw the title of your talk, I understood what the problem was.”

Colin says: As I listened to the Q&A session again, James framed this as a question of whether two instances allocated in different contexts should be considered the same or not. I think that might be useful framing to bring up early here. Gilad also pointed out that (paraphrasing) the BETA people knew an inner class was a property of the instance, not its own class.

discuss C# casts with reified generics

The key issues at work in the interaction of method-local inner classes and generics arise from the interaction of (1) nominal dynamic downcasts, (2) design choices about what runtime information about generic parameters is both persisted *and checked* by casts, and (3) (not) tracking all dependencies on type parameters in the type system. Essentially, Java and Scala both treat `MyFoo` as a sort of existential type — that there exists some `X` such that `MyFoo` extends `Foo<X>`. However, treating `MyFoo` as a closed rather than generic type is equivalent to assuming that there is only one existential for all instances — associated with the declaration site — rather than potentially different existentials for different instances. This combined with the fact that the JVM is unable to dynamically check for different instantiations leads to the unsoundness. Our dynamic solution resolves this with a specific implementation of casts on reified generics. Our static solution effectively prohibits the de facto existential capture of the enclosing type parameter.

The decision to treat types like the `MyFoo` variants in our examples as closed existential types that depend on other choices of types is not without parallels in other languages. Haskell and Rust, for example, include associated types [3], where a typeclass has a type as a member. A yet closer analogue is Scala's path-dependent types, where types may be members of an individual object, rather than associated with an individual class. The code below adapts the troublesome example method using Scala's type members:

```

trait Foo { type X; def x() : X; }
def m[Y](other: Any, newx: Y) : Foo2 =
  class MyFoo extends Foo { type X = Y; override def x() : X = newx; }
  newx match {
    case f: MyFoo => f
    case _ => MyFoo()
  }

```

This code compiles without warnings — and is not an issue. In this case the dependency of `MyFoo` instances on the type parameter used to call `m` is made explicit with a type member. But Scala now treats these

References

- 1 Ole Agesen, Stephen N Freund, and John C Mitchell. Adding type parameterization to the java language. In *Proceedings of the 12th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*, pages 49–65, 1997.
- 2 Nada Amin and Ross Tate. Java and scala's type systems are unsound: The existential crisis of null pointers. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2016*, page 838–848, New York, NY, USA, 2016. Association for Computing Machinery. doi: 10.1145/2983990.2984004.
- 3 Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005.

- 455 4 Andrew Kennedy and Don Syme. Design and implementation of generics for the .net common
456 language runtime. In *Proceedings of the ACM SIGPLAN 2001 conference on Programming*
457 *language design and implementation*, pages 1–12, 2001.
- 458 5 Martin Odersky, Enno Runne, and Philip Wadler. Two ways to bake your pizza—translating
459 parameterised types into java. In *Generic Programming: International Seminar on Generic*
460 *Programming Dagstuhl Castle, Germany, April 27–May 1, 1998 Selected Papers*, pages 114–132.
461 Springer, 2001.
- 462 6 Marco Servetto, Nick Webster, and Colin S. Gordon. Java method-local inner classes are
463 unsound. Talk at UNSOUND 2024, co-located with SPLASH 2024, October 2024. Recording
464 at <https://www.youtube.com/watch?v=Dfk0pk733dQ>. URL: [https://2024.splashcon.org/](https://2024.splashcon.org/details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound)
465 [details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound](https://2024.splashcon.org/details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound).